Prepared for
Leandro Schlottchauer
Andre Brener
Fuul.xyz

Prepared by
Sergio Corrales
Ignacio Sanmillan
Inversive Labs

January 21st 2026

ILAR-2026-01

Fuul Protocol - Solana

Smart Contract Security Assessment

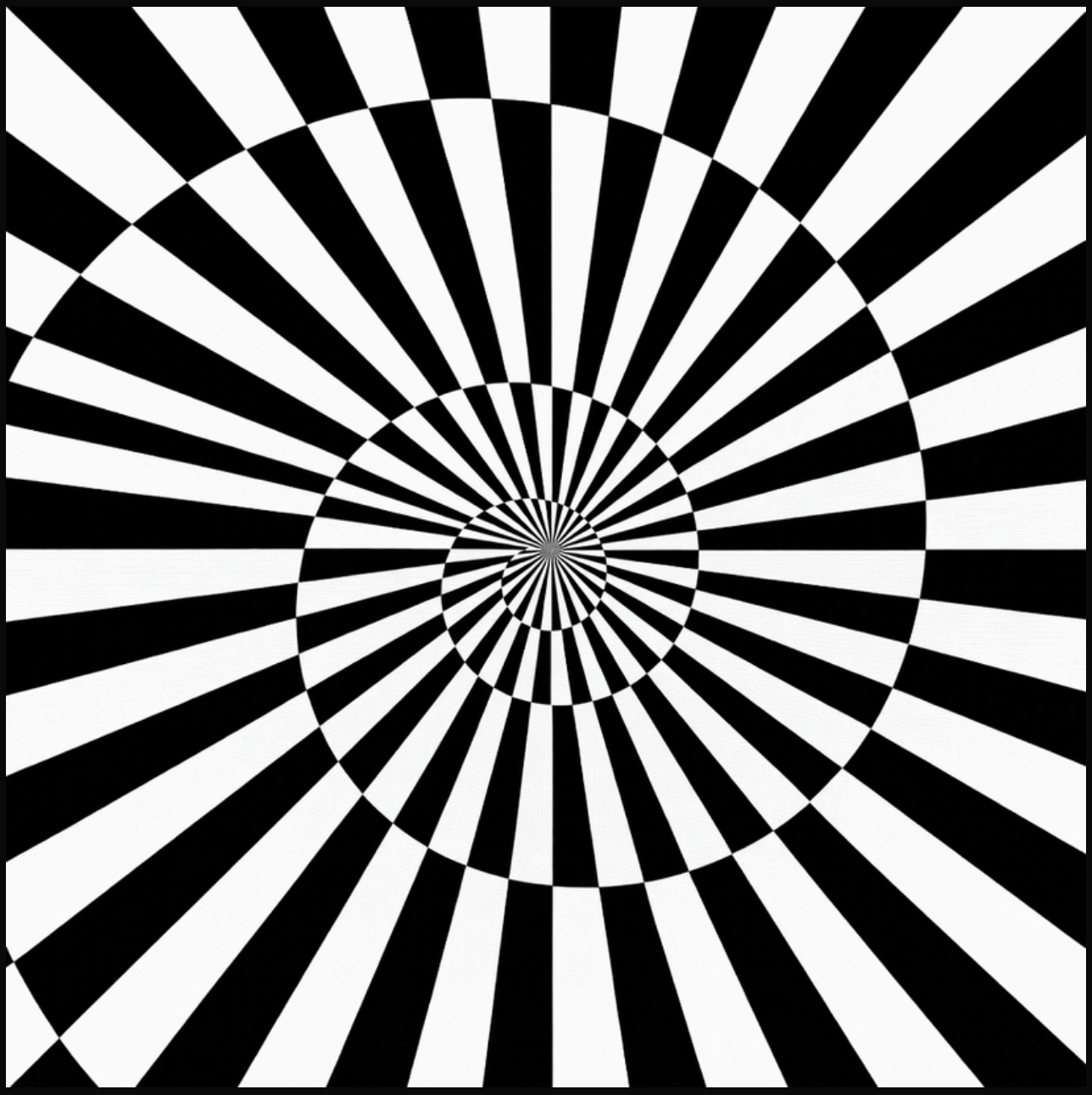


Table of Contents

1. About Inversive
2. Disclaimer
3. Risk Classification
4. Protocol Summary
5. Scope
6. Executive Summary
7. Summary of Findings
8. Detailed Findings
    - ILRV-2026-01.01: Ed25519 Multi-Signature Bypass
    - ILRV-2026-01.02: Proof Parameter Substitution Enables Replay Attacks and Attribution Fraud
    - ILRV-2026-01.03: Missing Validation Allows Removing Tokens With Active Project Budgets
    - ILRV-2026-01.04: Non-Canonical Token Accounts Cause Fund Locking and Budget Inconsistency
    - ILRV-2026-01.05: Missing Precondition Check: Token Must Be Inactive Before Removal
    - ILRV-2026-01.06: Rent Lamports Not Reclaimed on Token Removal
    - ILRV-2026-01.07: Fee Override Cannot Be Set Equal to Current Global Default
    - ILRV-2026-01.08: Unrestricted Global Configuration Initialization
    - ILRV-2026-01.09: Missing Checked Arithmetic in Budget Deduction
    - ILRV-2026-01.10: Global Cooldown Limit Causes Cross-Project Resource Contention
    - ILRV-2026-01.11: Missing Explicit Initialization of GlobalConfig Fields
    - ILRV-2026-01.12: Inefficient Early Return Logic in Fee Update Function
    - ILRV-2026-01.13: Missing Role Existence Check in revoke_project_role
    - ILRV-2026-01.14: Missing Global Role Check in renounce_global_role Instruction
    - ILRV-2026-01.15: Integer Division in Fee Calculation Causes Precision Loss and Revenue Leakage
9. Appendix: Proof of Concept Exploits

1. About Inversive Labs

Inversive Labs is a boutique security research firm dedicated to securing emerging blockchain ecosystems. We conduct security assessments for smart contracts and blockchain infrastructure, while also providing custom security tooling and tailored development services.

Prior to Inversive Labs, our founders established themselves as independent security researchers, discovering critical vulnerabilities in production protocols and contributing to the security of major DeFi and open-source projects. Our engineers bring diverse backgrounds in offensive security, low-level exploitation, and smart contract security.

Our expertise spans traditional security and blockchain attack vectors, enabling us to identify critical vulnerabilities across complex protocol implementations and contribute meaningful security research to rapidly evolving ecosystems.

For more on Inversive Labs' security research and services, visit https://inversive.xyz, reach us on X at @inversive_xyz or send us an email at contact@inversive.xyz

2. Disclaimer

This document presents the findings from an invariant testing campaign conducted on Fuul Protocol over a 1-week period. The testing focused on validating business logic invariants and economic properties through systematic fuzzing with Solaris, our stateful sBPF coverage-guided fuzzer.

This report documents vulnerabilities and invariant violations identified during the testing period. The analysis represents the state of the codebase at a specific point in time and does not account for subsequent modifications. Security testing is inherently limited by time, resources, and the evolving nature of attack vectors.

Our testing methodology adheres to industry best practices for security assessment and invariant validation. Findings are classified according to severity based on potential impact and exploitability. All identified issues have been verified through reproduction and root cause analysis.

This report is provided for informational purposes only and does not constitute investment advice, legal counsel, or a guarantee of security. Inversive Labs makes no warranties, express or implied, regarding the absence of vulnerabilities beyond those identified during the testing period. The deployment and use of the analyzed code remain at the sole risk and responsibility of the implementing parties.

3. Risk Classification

Our audit uses the following severity classification system:

SEVERITY	DESCRIPTION
CRITICAL	Issues that can lead to loss of funds, protocol insolvency, or complete system compromise. Immediate action required.
HIGH	Issues that can lead to significant loss of funds or major protocol disruption under certain conditions. Should be fixed promptly.
MEDIUM	Issues that can lead to unexpected behavior, minor loss of funds, or degraded user experience. Should be addressed.
LOW	Issues that represent potential risks or deviations from best practices. Recommended to fix.
INFORMATIONAL	Code quality improvements, gas optimizations, or suggestions for better practices. Optional fixes.

4. Protocol Summary

Fuul is a Solana-based protocol built with the Anchor framework that manages project budgets, token distributions, and multi-signature claim operations. The protocol enables:

- **Project Management:** Creation and administration of projects with role-based access control
- **Budget Management:** Deposit and withdrawal of fungible and non-fungible tokens
- **Claim System:** Multi-signature verification for token claims with cooldown limits
- **Currency Management:** Global token configuration with claim limits and cooldown periods
- **Fee System:** Configurable removal fees with global defaults and project-specific overrides

Key Components:

- Global configuration with role-based permissions
- Project-level budget tracking and token management
- Ed25519 signature verification for multi-party approvals
- Cooldown mechanism to prevent claim abuse
- Associated Token Account (ATA) integration

5. Scope

The audit covered the complete Fuul Protocol implementation. The repository was shared with Inversive Labs in a private capacity, and the entire codebase was in scope for security review.

Programs Tested

PROGRAM	COMMIT HASH	LINES OF CODE
fuul-protocol/protocol-program-svm	`d0454b40e6e08fc976797854d47e872228e9ae65`	~3,200

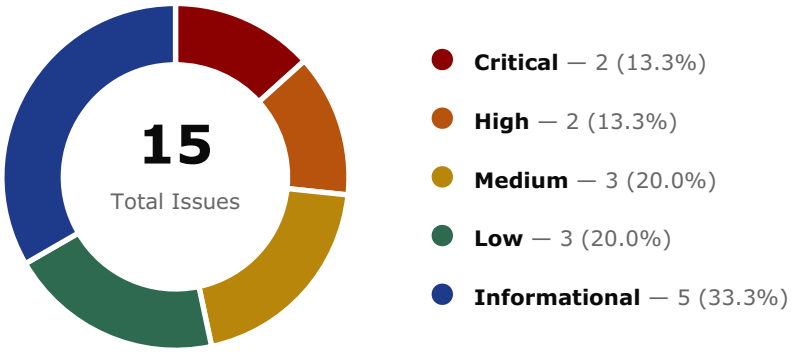
In Scope:

- **Solana On-Chain Program** (`programs/fuul-solana/`):
  - All instruction handlers (9 modules, 24+ instructions)
  - State account structures (6 account types)
  - Utility functions (signatures, transfers, access control)
  - Error definitions and constants
- **TypeScript SDK** (`packages/fuul-solana/`):
  - Full SDK implementation (`@wakeuplabs/fuul-solana`)
  - Instruction builders and transaction utilities
  - Account deserialization and PDA derivation
  - Message signing and verification utilities

6. Executive Summary

The audit of the Fuul Protocol identified **15 issues** across various severity levels. The most critical findings involve vulnerabilities in the multi-signature verification mechanism and claim system that could lead to unauthorized fund transfers and replay attacks.

The following graph summarizes the distribution of findings by severity:



Engagement Timeline

December 18, 2025
Initial meeting and scope definition

Kickoff

Audit Begins

December 22, 2025
Manual Security Audit commences

December 29, 2025
Initial findings report delivered

First Draft

Audit Fixes

Dec 30 - Jan 7, 2026
Protocol team implements remediation

Jan 8-15, 2026
Solaris fuzzing campaign

Invariant Testing
ILAR-2026-02

Testing Fixes

Jan 16-20, 2026
Address invariant testing findings

January 21, 2026
Final Reports and Testing Suite delivery

Final Report Delivery
ILAR-2026-01

7. Summary of Findings

ID	TITLE	SEVERITY	STATUS
ILRV-2026-01.01	Ed25519 Multi-Signature Bypass	CRITICAL	FIXED
ILRV-2026-01.02	Proof Parameter Substitution Enables Replay Attacks and Attribution Fraud	CRITICAL	FIXED
ILRV-2026-01.03	Missing Validation Allows Removing Tokens With Active Project Budgets	HIGH	ACK
ILRV-2026-01.04	Non-Canonical Token Accounts Cause Fund Locking and Budget Inconsistency	HIGH	FIXED
ILRV-2026-01.05	Missing Precondition Check: Token Must Be Inactive Before Removal	MEDIUM	FIXED
ILRV-2026-01.06	Rent Lamports Not Reclaimed on Token Removal	MEDIUM	ACK
ILRV-2026-01.07	Fee Override Cannot Be Set Equal to Current Global Default	MEDIUM	FIXED
ILRV-2026-01.08	Unrestricted Global Configuration Initialization	LOW	ACK
ILRV-2026-01.09	Missing Checked Arithmetic in Budget Deduction	LOW	FIXED
ILRV-2026-01.10	Global Cooldown Limit Causes Cross-Project Resource Contention	LOW	ACK
ILRV-2026-01.11	Missing Explicit Initialization of GlobalConfig Fields	INFORMATIONAL	FIXED
ILRV-2026-01.12	Inefficient Early Return Logic in Fee Update Function	INFORMATIONAL	FIXED
ILRV-2026-01.13	Missing Role Existence Check in revoke_project_role	INFORMATIONAL	FIXED
ILRV-2026-01.14	Missing Global Role Check in renounce_global_role Instruction	INFORMATIONAL	ACK
ILRV-2026-01.15	Integer Division in Fee Calculation Causes Precision Loss and Revenue Leakage	INFORMATIONAL	FIXED

Critical

[ILRV-2026-01.01] Ed25519 Multi-Signature Bypass

Summary

The multi-signature verification mechanism in the claim function can be bypassed by combining signatures from authorized signers who signed different messages. This allows an user to execute a claim with only one legitimate approval while satisfying the multi-signature threshold requirement by counting signatures from unrelated claims. This issue was addressed in commit `fc11b5d49b07d8d0a8e94ea860320be370dcf458`.

Description

The protocol requires multiple authorized signers to approve each claim operation before execution. The vulnerability exists in the verify_ed25519_signature function programs/fuul-solana/src/utils/signatures.rs:183-194, which extracts the message from only the first signature entry while counting all signers from all entries, without validating that all signatures reference the same message.

Relevant code:

```
pub fn verify_ed25519_signature(ix_sysvar_account: &AccountInfo) -> Result<(Vec<Pubkey>, Vec<u8>)> {
    let offsets_vec = parse_ed25519_ix_data(&ed_ix.data)?;

    // Extracts message ONLY from first signature entry
    let message = extract_signed_message(&ed_ix.data, &offsets_vec[0]).to_vec();

    // Counts ALL signers from ALL entries without validating they signed the same message
    for offsets in offsets_vec.iter() {
        let pubkey = extract_signer_pubkey(&ed_ix.data, offsets)?;
        if seen.insert(pubkey) {
            signers.push(pubkey);
        }
    }
    Ok((signers, message))
}
```

The function returns the message from the first entry and a list of all signers, but never validates that all signature entries reference identical message data.

Root cause: The Solana Ed25519 precompile validates each signature independently against its specified message. Each signature entry in the Ed25519 instruction data contains its own `message_data_offset` and `message_data_size` fields, allowing different signatures within the same instruction to verify different messages. The precompile does not enforce message uniformity across signature entries—it only validates that each individual signature is cryptographically valid for its specified message.

The program incorrectly assumes that the Ed25519 precompile guarantees all signatures verify the same message, as reflected in the comment "All signatures are already verified by the Ed25519 program and sign the same message". This assumption is false.

Exploitation path: Consider a configuration with `required_signers_for_claim = 2`:

1. Signer A creates and signs ClaimMessage A: `{ amount: 1_000_000, recipient: A, project: ProjectX }`
2. Signer B independently creates and signs ClaimMessage B: `{ amount: 500_000, recipient: B, project: ProjectX }`
3. A malicious actor constructs an Ed25519 instruction containing both signatures, where:
    - Signature entry 0 references ClaimMessage A at offset 126
    - Signature entry 1 references ClaimMessage B at offset 226 (different message)

4. The Ed25519 precompile validates both signatures independently and returns SUCCESS
5. `verify_ed25519_signature` extracts ClaimMessage A from entry 0 but counts both signers A and B
6. The signer count validation passes: `2 >= required_signers_for_claim`
7. ClaimMessage A executes, transferring 1,000,000 tokens to recipient A

Signer B is counted as an approver despite never reviewing or approving ClaimMessage A. Signer B only signed their own independent claim for a different amount and recipient.

Impact

The multi-signature authorization control is completely bypassed. An attacker can execute claims with only one legitimate approval by reusing signatures from unrelated claims to satisfy the threshold requirement. This enables unauthorized fund transfers without proper multi-party review, potentially draining entire project budgets.

Recommendation

Validate that all signature entries reference identical message data in programs/fuul-solana/src/utils/signatures.rs:

File: `programs/fuul-solana/src/utils/signatures.rs` (after line 184)

```
 pub fn verify_ed25519_signature(ix_sysvar_account: &AccountInfo) -> Result<(Vec<Pubkey>, Vec<u8>)> {
     let offsets_vec = parse_ed25519_ix_data(&ed_ix.data)?;
     require!(!offsets_vec.is_empty(), FuulError::InvalidInstructionSysvar);

     let message = extract_signed_message(&ed_ix.data, &offsets_vec[0]).to_vec();
+
+    // Validate all signatures reference the same message
+    let first_msg_offset = offsets_vec[0].message_data_offset;
+    let first_msg_size = offsets_vec[0].message_data_size;
+
+    for offsets in offsets_vec.iter().skip(1) {
+        require!(
+            offsets.message_data_offset == first_msg_offset &&
+            offsets.message_data_size == first_msg_size,
+            FuulError::InvalidInstructionSysvar
+        );
+    }

     let mut signers = Vec::with_capacity(offsets_vec.len());
     let mut seen = HashSet::with_capacity(offsets_vec.len());
     for offsets in offsets_vec.iter() {
         let pubkey = extract_signer_pubkey(&ed_ix.data, offsets)?;
         if seen.insert(pubkey) {
             signers.push(pubkey);
         }
     }

     Ok((signers, message))
 }
```

This ensures all signature entries have identical `message_data_offset` and `message_data_size`, guaranteeing all signers approved the same claim.

Proof of Concept: See Appendix: ILRV-2026-01.01 PoC for a working exploit demonstration.

[ILRV-2026-01.02] Proof Parameter Substitution Enables Replay Attacks and Attribution Fraud

Summary

The claim function fails to validate that the `proof` and `proof_without_project` instruction parameters match the values in the signed message. This allows a single signed authorization to be reused unlimited times with different proof values, bypassing replay protection and enabling complete budget drainage. This issue was addressed in commit `31c1a9e4f5aa152d653e22d387a878db38ce1ded`.

Description

The claim function implements replay protection using a PDA with seeds `[PROJECT_ATTRIBUTION_TAG, project, proof]` where the `init` constraint prevents duplicate claims. However, the `proof` parameter used in the PDA derivation comes from the instruction parameters and is never validated against the signed message's `claim.data.proof` field. This allows the same signature to be used with different proof values, creating different PDAs each time and bypassing replay protection entirely.

Relevant code programs/fuul-solana/src/instructions/project_budget.rs:652-694:

```
pub fn claim(&mut self, project_nonce: u64, proof: [u8; 32], proof_without_project: [u8; 32]) -> Result<()> {
    let claim = ClaimMessage::try_from_slice(&message)?;

    // Validates accounts match signed message
    require!(project.key() == claim.data.project, FuulError::SignedMessageMismatch);
    require!(self.recipient.key() == claim.data.recipient, FuulError::SignedMessageMismatch);
    require!(self.token_mint.key() == claim.data.token_mint, FuulError::SignedMessageMismatch);

    // Validates hash relationship of function parameters
    require!(
        keccak(&[proof_without_project.as_slice(), project.key().as_ref()].concat()).to_bytes() == proof,
        FuulError::InvalidProof
    );

    // Never validates proof == claim.data.proof
    // Never validates proof_without_project == claim.data.proof_without_project

    // Stores function parameter (not signed value) in nullifier
    self.project_attribution.set_inner(ProjectAttribution {
        proof,  // Uses unvalidated function parameter
        // ...
    });
}
```

PDA constraint lines 367-378:

```
#[account(
    init,
    seeds = [
        PROJECT_ATTRIBUTION_TAG.as_bytes(),
        project.key().as_ref(),
        proof.as_ref(),  // Uses unvalidated function parameter
    ],
    bump
)]
pub project_attribution: Box<Account<'info, ProjectAttribution>>,
```

Root cause: The function validates that instruction parameters satisfy the hash relationship `keccak(proof_without_project + project) == proof`, but never validates that these parameters match the signed message fields `claim.data.proof` and `claim.data.proof_without_project`. Since the PDA seeds use the unvalidated function parameter, different proof values create different PDAs, allowing the same signature to be reused unlimited times.

Exploitation path: Consider a user authorized to claim 1,000 USDC from a project with 10,000 USDC budget:

1. Backend signs ClaimMessage with `proof = keccak(affiliate_A + project)`
2. User submits claim #1 with `proof = keccak(affiliate_B + project)` (different value)
   - Signature validates (signed the original message)
   - Hash relationship validates (function parameters are self-consistent)
   - PDA `[TAG, project, keccak(affiliate_B)]` doesn't exist → init succeeds
   - User receives 1,000 USDC, budget reduced to 9,000 USDC
3. User submits claim #2 with `proof = keccak(affiliate_C + project)` (different value)
   - Same signature still valid
   - New PDA `[TAG, project, keccak(affiliate_C)]` → init succeeds
   - User receives another 1,000 USDC, budget reduced to 8,000 USDC
4. User repeats 8 more times with different proof values
5. User drains entire 10,000 USDC budget using one signature

Each claim creates a new ProjectAttribution account with a different (arbitrary) affiliate ID, also enabling attribution fraud.

Impact

A single signed authorization can be reused to drain the entire project budget. The only limits are budget depletion and signature deadline expiration. This completely breaks the one-signature-per-claim security model and enables systematic fund theft at scale. Additionally, each replayed claim credits an arbitrary affiliate, corrupting all attribution data and enabling commission theft.

Recommendation

Validate that instruction parameters match the signed message in programs/fuul-solana/src/instructions/project_budget.rs:

File: programs/fuul-solana/src/instructions/project_budget.rs (after line 676)

```
pub fn claim(&mut self, project_nonce: u64, proof: [u8; 32], proof_without_project: [u8; 32]) -> Result<()> {
    // ... signature verification ...
    let claim = ClaimMessage::try_from_slice(&message)?;

    // verify accounts provided match the message data
    require!(project.key() == claim.data.project, FuulError::SignedMessageMismatch);
    require!(self.recipient.key() == claim.data.recipient, FuulError::SignedMessageMismatch);
    require!(self.token_mint.key() == claim.data.token_mint, FuulError::SignedMessageMismatch);
+
+    // Validate proof parameters match signed message
+    require!(proof == claim.data.proof, FuulError::SignedMessageMismatch);
+    require!(proof_without_project == claim.data.proof_without_project, FuulError::SignedMessageMismatch);

    // Validate message domain, deadline, program id, etc
    validate_message_domain(&claim.domain)?;

    // Validate proof is for the project
    require!(
        keccak(&[proof_without_project.as_slice(), project.key().as_ref()].concat()).to_bytes() == proof,
        FuulError::InvalidProof
    );
```

This ensures only the signed proof value can be used, creating one specific PDA and preventing replay attacks.

Proof of Concept: See Appendix: ILRV-2026-01.02 PoC for a working exploit demonstration.

High

[ILRV-2026-01.03] Missing Validation Allows Removing Tokens With Active Project Budgets

Summary

The `remove_currency_token` instruction does not validate whether projects have active budgets denominated in the token being removed, allowing an admin to deactivate a token while funds remain locked in project budgets.

Description

The protocol documentation specifies that remove_currency_token "Cannot be used if projects have active budgets," but this validation is completely missing from the implementation. The function only sets `is_active = false` without checking if any projects hold budgets in that token.

Relevant code programs/fuul-solana/src/instructions/admin_currencies.rs:184-194:

```
impl<'info> RemoveCurrencyToken<'info> {
    pub fn remove_currency_token(&mut self) -> Result<()> {
        self.currency_token.is_active = false;  // Only sets inactive flag

        // No check for active project budgets

        emit!(LogCurrencyTokenRemovedEvent {
            token_mint: self.token_mint.key(),
        });
        Ok(())
    }
}
```

Root cause: The function lacks any mechanism to track or validate whether projects have non-zero budgets for the token being removed. The `CurrencyToken` struct does not maintain a count of projects with active budgets, and the function performs no iteration or validation before deactivation.

Exploitation path:

1. Project A deposits 100,000 USDC into their budget
2. Project B deposits 50,000 USDC into their budget
3. Admin calls `remove_currency_token` for USDC token
4. Function succeeds, setting `currency_token.is_active = false`
5. Projects A and B cannot deposit more USDC (deposits require `is_active = true`)
6. Existing 150,000 USDC remains locked in project budgets
7. Claims can still execute and drain budgets, but projects cannot replenish funds
8. System enters inconsistent state where inactive token has active budgets

Impact

Tokens can be deactivated while funds remain locked in project budgets, creating an inconsistent state where projects cannot deposit additional funds but existing budgets remain accessible for claims. This breaks the protocol's invariant that inactive tokens should have no active budgets and can lead to operational issues and fund lockup.

Recommendation

Add an `active_project_count` field to track projects with non-zero budgets and validate before removal:

Step 1: Update `CurrencyToken` struct

File: programs/fuul-solana/src/state/currency_token.rs

```
 #[account]
 #[derive(InitSpace)]
 pub struct CurrencyToken {
     pub is_initialized: bool,
     pub token_mint: Pubkey,
     pub token_type: TokenType,
     pub is_active: bool,
     pub claim_limit_per_cooldown: u64,
     pub cumulative_claim_per_cooldown: u64,
     pub claim_cooldown_period_started: i64,
+    pub active_project_count: u32,
 }
```

Step 2: Update budget tracking in deposit/remove functions

File: programs/fuul-solana/src/instructions/project_budget.rs

```
 // In deposit_fungible_token and deposit_non_fungible_token:
 pub fn deposit_fungible_token(&mut self, amount: u64) -> Result<()> {
+    // Increment counter when budget goes from 0 to non-zero
+    if self.project_currency_budget.budget == 0 {
+        self.currency_token.active_project_count =
+            self.currency_token.active_project_count.checked_add(1)?;
+    }
+
     self.project_currency_budget.budget = self.project_currency_budget.budget
         .checked_add(amount)?;
     // ...
 }

 // In remove_fungible_token and remove_non_fungible_token:
 pub fn remove_fungible_token(&mut self, amount: u64) -> Result<()> {
     self.project_currency_budget.budget = self.project_currency_budget.budget
         .checked_sub(amount)?;
+
+    // Decrement counter when budget reaches 0
+    if self.project_currency_budget.budget == 0 {
+        self.currency_token.active_project_count =
+            self.currency_token.active_project_count.checked_sub(1)?;
+    }
     // ...
 }
```

Step 3: Add validation to `remove_currency_token`

File: programs/fuul-solana/src/instructions/admin_currencies.rs

```
 pub fn remove_currency_token(&mut self) -> Result<()> {
+    // Validate no projects have active budgets
+    require!(
+        self.currency_token.active_project_count == 0,
+        FuulError::TokenHasActiveProjects
+    );
+
     self.currency_token.is_active = false;

     emit!(LogCurrencyTokenRemovedEvent {
         token_mint: self.token_mint.key(),
     });

     Ok(())
 }
```

[ILRV-2026-01.04] Non-Canonical Token Accounts Cause Fund Locking and Budget Inconsistency

Summary

The deposit and remove functions validate token accounts using only `owner` and `mint` constraints, while the claim function enforces canonical Associated Token Accounts (ATA) using `associated_token` constraints. This inconsistency allows deposits to non-canonical accounts where funds become inaccessible, and withdrawals to non-canonical accounts where funds may be lost. This issue was addressed in commit `85d8f5deff2739123090027e356800cb18ea9f85`.

Description

Solana allows multiple token accounts with the same owner and mint. The deposit/remove functions accept any token account matching these criteria, but the claim function only accesses the deterministic canonical ATA derived from `get_associated_token_address(owner, mint)`. This creates a mismatch where funds can be deposited to accounts that claims cannot access.

Relevant code (deposit/remove functions):

```rust
// Deposit functions - only validate owner and mint
#[account(
    mut,
    constraint = project_ata.owner == project.key() @FuulError::Unauthorized,
    constraint = project_ata.mint == token_mint.key() @FuulError::InvalidTokenMint,
)]
pub project_ata: Option<Account<'info, TokenAccount>>,
```

Secure code (claim function):

```rust
// Claim function - enforces canonical ATA
#[account(
    mut,
    associated_token::mint = token_mint,
    associated_token::authority = project
)]
pub project_ata: Option<Account<'info, TokenAccount>>,
```

Root cause: The deposit/remove functions accept any token account with matching `owner` and `mint`, allowing multiple accounts per owner-mint pair. The claim function uses `associated_token` constraints that derive and enforce the single canonical ATA address. This mismatch means deposits can go to accounts that claims cannot access.

Exploitation path:

1. User A deposits 1,000 USDC to canonical ATA
   - `canonical_ata.balance = 1,000 USDC`
   - `project_currency_budget.budget = 1,000 USDC`
2. User B creates non-canonical token account with same owner and mint
3. User B deposits 500 USDC to non-canonical account
   - Validation passes (owner and mint match)
   - `non_canonical_ata.balance = 500 USDC`
   - `project_currency_budget.budget = 1,500 USDC` (incremented)
4. User C attempts to claim 1,500 USDC
   - Budget check passes (1,500 <= 1,500)
   - Claim accesses only canonical ATA (has 1,000 USDC)
   - Transfer fails with "Insufficient funds"
5. Result: 500 USDC locked in non-canonical account, budget shows incorrect available balance

Impact

Funds deposited to non-canonical token accounts become permanently inaccessible through the normal claim flow, while the budget incorrectly reflects these funds as available. This causes claim failures with "Insufficient funds" errors despite sufficient budget, creates operational overhead for manual recovery, and enables low-cost griefing attacks where minimal deposits to non-canonical accounts disrupt the protocol.

Recommendation

Replace `owner` and `mint` constraint validations with `associated_token` constraints to enforce canonical ATA usage:

File: programs/fuul-solana/src/instructions/project_budget.rs

For deposit_fungible_token **(lines 72-78):**

```
 /// The project token account to deposit to.
 #[account(
     mut,
-    constraint = project_ata.owner == project.key() @FuulError::Unauthorized,
-    constraint = project_ata.mint == token_mint.key() @FuulError::InvalidTokenMint,
+    associated_token::mint = token_mint,
+    associated_token::authority = project
 )]
 pub project_ata: Option<Account<'info, TokenAccount>>,
```

For deposit_non_fungible_token **(lines 123-129):**

```
 /// The project token account to deposit to (the NFT destination)
 #[account(
     mut,
-    constraint = project_ata.owner == project.key() @FuulError::Unauthorized,
-    constraint = project_ata.mint == token_mint.key() @FuulError::InvalidTokenMint,
+    associated_token::mint = token_mint,
+    associated_token::authority = project
 )]
 pub project_ata: Account<'info, TokenAccount>>,
```

For remove_fungible_token **(lines 190-211):**

```
 /// The authority token account to withdraw to
 #[account(
     mut,
-    constraint = authority_ata.owner == authority.key() @FuulError::InvalidAtaOwner,
-    constraint = authority_ata.mint == token_mint.key() @FuulError::InvalidTokenMint,
+    associated_token::mint = token_mint,
+    associated_token::authority = authority
 )]
 pub authority_ata: Option<Account<'info, TokenAccount>>,

 /// The project currency vault ata to withdraw from
 #[account(
     mut,
-    constraint = project_ata.owner == project.key() @FuulError::InvalidAtaOwner,
-    constraint = project_ata.mint == token_mint.key() @FuulError::InvalidTokenMint,
+    associated_token::mint = token_mint,
+    associated_token::authority = project
 )]
 pub project_ata: Option<Account<'info, TokenAccount>>,
```

For remove_non_fungible_token **(lines 244-267):**

```
 /// The authority token account to withdraw to (the NFT destination)
 #[account(
     mut,
-    constraint = authority_ata.owner == authority.key() @FuulError::Unauthorized,
-    constraint = authority_ata.mint == token_mint.key() @FuulError::InvalidTokenMint,
+    associated_token::mint = token_mint,
+    associated_token::authority = authority
 )]
 pub authority_ata: Account<'info, TokenAccount>>,

 /// The project token account to withdraw from (the NFT source)
 #[account(
     mut,
-    constraint = project_ata.owner == project.key() @FuulError::Unauthorized,
-    constraint = project_ata.mint == token_mint.key() @FuulError::InvalidTokenMint,
+    associated_token::mint = token_mint,
+    associated_token::authority = project,
     constraint = project_ata.amount >= 1 @FuulError::InsufficientBalance,
 )]
 pub project_ata: Account<'info, TokenAccount>>,
```

For claim fee_collector_ata **(lines 320-325):**

```
 /// The fee collector account
 #[account(
     mut,
-    constraint = fee_collector_ata.owner == fee_collector.key() @FuulError::InvalidAtaOwner,
-    constraint = fee_collector_ata.mint == token_mint.key() @FuulError::InvalidTokenMint,
+    associated_token::mint = token_mint,
+    associated_token::authority = fee_collector
 )]
 pub fee_collector_ata: Option<Account<'info, TokenAccount>>,
```

This ensures all functions use the same canonical ATA, preventing fund locking and maintaining budget accuracy.

Medium

[ILRV-2026-01.05] Missing Precondition Check: Token Must Be Inactive Before Removal

Summary

The `remove_currency_token` instruction does not validate that the token is already inactive before allowing removal, bypassing the intended two-step removal process (deactivate → remove) documented in the protocol specification. This issue was addressed in commit `6fe85e56106ca5664c51b52b3c7e88ca19110ce1`.

Description

The protocol documentation specifies that the token Account must be inactive as a precondition for removal, but the implementation only sets `is_active = false` without checking if it was already inactive. This allows admins to call `remove_currency_token` on active tokens, effectively using it as a deactivation function rather than a removal function.

Relevant code programs/fuul-solana/src/instructions/admin_currencies.rs:184-194**:**

```
impl<'info> RemoveCurrencyToken<'info> {
    pub fn remove_currency_token(&mut self) -> Result<()> {
        // No check that token is already inactive

        self.currency_token.is_active = false;  // Just sets it to false

        emit!(LogCurrencyTokenRemovedEvent {
            token_mint: self.token_mint.key(),
        });
        Ok(())
    }
}
```

Root cause: The function treats `is_active = false` as an effect rather than validating it as a precondition. The documentation implies a two-step process: (1) deactivate via `update_currency_token`, then (2) remove via `remove_currency_token`. The missing validation allows step 1 to be skipped.

Exploitation path:

1. Admin intends to temporarily deactivate USDC token
2. Admin accidentally calls `remove_currency_token(USDC)` instead of `update_currency_token`
3. Function succeeds without error (expected: "Token must be inactive first")
4. Token is deactivated but not removed
5. Admin believes token was removed but it remains in the system

Impact

The missing validation bypasses the intended two-step safety process, reducing operational safety and creating confusing function behavior where `remove_currency_token` acts as a deactivation function for active tokens but a removal function for inactive tokens. This inconsistency with documentation can lead to operational errors and makes it harder to enforce proper token lifecycle management.

Recommendation

Add validation to ensure the token is already inactive before allowing removal:

File: programs/fuul-solana/src/instructions/admin_currencies.rs (lines 184-194)

```
 impl<'info> RemoveCurrencyToken<'info> {
     pub fn remove_currency_token(&mut self) -> Result<()> {
+        // Validate token is already inactive
+        require!(
+            !self.currency_token.is_active,
+            FuulError::TokenMustBeInactive
+        );
+
         // Additional validations (see H-01)
         // Close account and return rent (see M-02)

         emit!(LogCurrencyTokenRemovedEvent {
             token_mint: self.token_mint.key(),
         });
         Ok(())
     }
 }
```

This enforces the two-step process: admins must first deactivate via `update_currency_token`, then remove via `remove_currency_token`.

[ILRV-2026-01.06] Rent Lamports Not Reclaimed on Token Removal

Summary

The `remove_currency_token` instruction does not close the currency token account or reclaim rent lamports, despite the protocol documentation stating it "Closes a currency token account" and "Transfers rent to authority."

Description

The protocol documentation specifies that remove_currency_token should transfer rent to the authority and close the account. However, the implementation only sets `is_active = false`, leaving the account and its rent-exempt balance (~0.002 SOL) in the protocol permanently.

Relevant code programs/fuul-solana/src/instructions/admin_currencies.rs:63-81 **and** Lines 184-194**:**

```
#[derive(Accounts)]
pub struct RemoveCurrencyToken<'info> {
    #[account(
        mut,
        // Missing:close = authority constraint
        seeds = [CURRENCY_TOKEN_TAG.as_bytes(), token_mint.key().as_ref()],
        bump
    )]
    pub currency_token: Account<'info, CurrencyToken>,
    // ...
}

impl<'info> RemoveCurrencyToken<'info> {
    pub fn remove_currency_token(&mut self) -> Result<()> {
        self.currency_token.is_active = false;  // Only sets flag

        // Missing: Account closure logic

        emit!(LogCurrencyTokenRemovedEvent {
            token_mint: self.token_mint.key(),
        });
        Ok(())
    }
}
```

Root cause: The account struct lacks the `close = authority` constraint that would automatically transfer all lamports to the authority and zero out the account data. Without this constraint, the account remains in the system indefinitely with its rent-exempt balance locked.

Impact: Each removed token leaves ~0.002 SOL permanently locked in the protocol. Over time, this accumulates as tokens are added and removed. The function is also misnamed—"remove" implies deletion, but it only deactivates the token.

Recommendation

Add the `close = authority` constraint to reclaim rent on removal:

File: programs/fuul-solana/src/instructions/admin_currencies.rs (lines 63-81)

```
 #[derive(Accounts)]
 pub struct RemoveCurrencyToken<'info> {
     #[account(mut, signer)]
     pub authority: Signer<'info>,

     #[account(mut, address = GLOBAL_CONFIG_ADDRESS)]
     pub global_config: Account<'info, GlobalConfig>,

     #[account(
         mut,
+        close = authority,
         seeds = [CURRENCY_TOKEN_TAG.as_bytes(), token_mint.key().as_ref()],
         bump
     )]
     pub currency_token: Account<'info, CurrencyToken>,

     /// CHECK: This is the mint address used only for PDA derivation
     pub token_mint: UncheckedAccount<'info>,

     pub system_program: Program<'info, System>,
 }
```

With this change, the `close` constraint automatically: - Transfers all lamports from the account to `authority` - Zeros out the account data - Marks the account as closed

The handler can be simplified since account closure is handled automatically:

```
impl<'info> RemoveCurrencyToken<'info> {
    pub fn remove_currency_token(&mut self) -> Result<()> {
        // Add validations (see H-01, M-01)
-       self.currency_token.is_active = false;

        emit!(LogCurrencyTokenRemovedEvent {
            token_mint: self.token_mint.key(),
        });
        Ok(())
    }
}
```

[ILRV-2026-01.07] Fee Override Cannot Be Set Equal to Current Global Default

Summary

The `update_project_fees` function compares new fee values against the effective fee (via getter methods) instead of the stored value, preventing admins from setting project-specific overrides when the desired value equals the current global default. This issue was addressed in commit `028c2a91e68dd9ba3e9d7b01168344d76fc6f502`.

Description

Project fee management uses `Option<u64>` fields where `None` means "use global default" and `Some(value)` means "use this specific override." The getter methods implement fallback logic that returns the global default when the project-specific value is `None`.

Relevant code programs/fuul-solana/src/instructions/project.rs:90-96**:**

```
if let Some(user_native_claim_fee) = user_native_claim_fee {
    // Compares against effective value (getter with fallback)
    if user_native_claim_fee == self.project.get_user_native_claim_fee(&self.global_config) {
        return Err(FuulError::NoNewChanges.into());
    }
    self.project.fee_management.user_native_claim_fee = Some(user_native_claim_fee);
}
```

Getter method with fallback:

```
pub fn get_user_native_claim_fee(&self, global_config: &GlobalConfig) -> u64 {
    self.fee_management.user_native_claim_fee.unwrap_or(global_config.fee_management.user_native_claim_fee)
}
```

Root cause: The function compares the new value against the effective value returned by the getter (which includes fallback logic) instead of comparing against the stored `Option<u64>` value. This makes it impossible to transition from `None` (use global) to `Some(X)` when X equals the current global default.

Exploitation path:

1. Initial state: `global_config.user_native_claim_fee = 1000`, `project.user_native_claim_fee = None`
2. Admin wants to lock in current fee before raising global fee (grandfathering)
3. Admin calls `update_project_fees(user_native_claim_fee: Some(1000))`
4. Validation: `1000 == get_user_native_claim_fee()` → `1000 == None.unwrap_or(1000)` → `1000 == 1000` → TRUE

5. Transaction fails with `NoNewChanges` error
6. Result: Cannot set explicit override to lock in current fee

The same issue affects `project_claim_fee` and `remove_fee` fields.

Impact

Admins cannot set project-specific fee overrides when the desired value equals the current global default. This breaks grandfathering functionality where protocols need to lock in current fees for existing projects before raising global fees. The function cannot distinguish between `None` (follow global changes) and `Some(X)` (locked value).

Recommendation

Compare against the stored `Option<u64>` value instead of the effective value from the getter:

File: programs/fuul-solana/src/instructions/project.rs

For user_native_claim_fee **(line 91):**

```
 if let Some(user_native_claim_fee) = user_native_claim_fee {
-    if user_native_claim_fee == self.project.get_user_native_claim_fee(&self.global_config) {
+    if Some(user_native_claim_fee) == self.project.fee_management.user_native_claim_fee {
         return Err(FuulError::NoNewChanges.into());
     }
     self.project.fee_management.user_native_claim_fee = Some(user_native_claim_fee);
 }
```

For project_claim_fee **(line 101):**

```
 if let Some(project_claim_fee) = project_claim_fee {
-    if project_claim_fee == self.project.get_project_claim_fee(&self.global_config) {
+    if Some(project_claim_fee) == self.project.fee_management.project_claim_fee {
         return Err(FuulError::NoNewChanges.into());
     }

     if project_claim_fee > BASIS_POINTS {
         return Err(FuulError::InvalidPercentage.into());
     }

     self.project.fee_management.project_claim_fee = Some(project_claim_fee);
 }
```

For remove_fee **(line 115):**

```
 if let Some(remove_fee) = remove_fee {
-    if remove_fee == self.project.get_remove_fee(&self.global_config) {
+    if Some(remove_fee) == self.project.fee_management.remove_fee {
         return Err(FuulError::NoNewChanges.into());
     }

     if remove_fee > BASIS_POINTS {
         return Err(FuulError::InvalidPercentage.into());
     }

     self.project.fee_management.remove_fee = Some(remove_fee);
 }
```

This allows the function to detect actual changes in the stored value, enabling transitions from `None` to `Some(X)` regardless of X's value.

Low

[ILRV-2026-01.08] Unrestricted Global Configuration Initialization

Summary

The `create_global_config` instruction allows any user to initialize the `GlobalConfig` account and receive all global admin roles, with no restriction on who can be the initial authority.

Description

The `create_global_config` instruction uses `init_if_needed` with a fixed PDA address, allowing anyone to call it. The first caller to successfully initialize the account receives all global roles (Admin, Pauser, Unpauser, Signer).

Relevant code programs/fuul-solana/src/instructions/create_global_config.rs:9-27**:**

```
#[derive(Accounts)]
pub struct CreateGlobalConfig<'info> {
    #[account(signer, mut)]
    pub authority: Signer<'info>,  // No restriction on who can be authority

    #[account(
        init_if_needed,
        address = GLOBAL_CONFIG_ADDRESS,  // Fixed PDA address
        seeds = [GLOBAL_CONFIG_TAG.as_bytes()],
        bump,
        payer = authority,
        space = GlobalConfig::DISCRIMINATOR.len() + GlobalConfig::INIT_SPACE,
    )]
    pub global_config: Account<'info, GlobalConfig>,
    // ...
}
```

Handler grants all roles to caller:

```
self.global_config.roles_mapping.grant_role(self.authority.key(), GlobalRole::Admin)?;
self.global_config.roles_mapping.grant_role(self.authority.key(), GlobalRole::Pauser)?;
self.global_config.roles_mapping.grant_role(self.authority.key(), GlobalRole::Unpauser)?;
self.global_config.roles_mapping.grant_role(self.authority.key(), GlobalRole::Signer)?;
```

Root cause: No access control on the `authority` signer. While the fixed PDA address and `init_if_needed` prevent multiple instances, there's a race condition where any user can front-run the legitimate deployer to gain admin control.

Impact: If an unauthorized user front-runs the deployment, they gain complete control of the protocol. The legitimate team would need to redeploy the entire program with a different program ID.

Recommendation

Restrict the `authority` to a hardcoded trusted public key:

File: programs/fuul-solana/src/instructions/create_global_config.rs

```
+use anchor_lang::solana_program::pubkey;
+
+const INITIAL_ADMIN: Pubkey = pubkey!("YourTrustedAdminPublicKeyHere");
+
 #[derive(Accounts)]
 pub struct CreateGlobalConfig<'info> {
-    #[account(signer, mut)]
+    #[account(
+        signer,
+        mut,
+        address = INITIAL_ADMIN @ FuulError::Unauthorized
+    )]
     pub authority: Signer<'info>,

     #[account(
         init_if_needed,
         address = GLOBAL_CONFIG_ADDRESS,
         seeds = [GLOBAL_CONFIG_TAG.as_bytes()],
         bump,
         payer = authority,
         space = GlobalConfig::DISCRIMINATOR.len() + GlobalConfig::INIT_SPACE,
     )]
     pub global_config: Account<'info, GlobalConfig>,

     pub system_program: Program<'info, System>,
 }
```

Alternatively, ensure program deployment and config initialization happen atomically in the same transaction, or change `init_if_needed` to `init` to make the race condition more explicit.

[ILRV-2026-01.09] Missing Checked Arithmetic in Budget Deduction

Summary

The `claim` function performs unchecked addition before checked subtraction, potentially causing silent overflow in the intermediate calculation. This issue was addressed in commit `9912233dbddb9b9833159bf34ce7b30e89c348d0`.

Description

The budget deduction logic adds `claim.data.amount + project_claim_fee_amount` without checked arithmetic before passing the result to `checked_sub`.

Relevant code programs/fuul-solana/src/instructions/project_budget.rs**:**

```rust
self.project_currency_budget.budget
    .checked_sub(claim.data.amount + project_claim_fee_amount)  // Unchecked addition
    .ok_or(FuulError::Underflow)?;
```

Root cause: The addition `claim.data.amount + project_claim_fee_amount` uses the `+` operator which can overflow silently, wrapping around to a small value. This would cause the `checked_sub` to succeed with an incorrect deduction amount.

Impact: If the addition overflows, the budget would be reduced by the wrapped value instead of the actual total, allowing more claims than the budget should permit.

Recommendation

Use checked arithmetic for the addition:

File: programs/fuul-solana/src/instructions/project_budget.rs (claim function)

```
-self.project_currency_budget.budget = self.project_currency_budget.budget
-    .checked_sub(claim.data.amount + project_claim_fee_amount)
-    .ok_or(FuulError::Underflow)?;
+let total_to_deduct = claim.data.amount
+    .checked_add(project_claim_fee_amount)
+    .ok_or(FuulError::Overflow)?;
+
+self.project_currency_budget.budget = self.project_currency_budget.budget
+    .checked_sub(total_to_deduct)
+    .ok_or(FuulError::Underflow)?;
```

[ILRV-2026-01.10] Global Cooldown Limit Causes Cross-Project Resource Contention

Summary

The claim cooldown mechanism uses a global limit shared across all projects using the same token, causing legitimate projects to compete for the same resource. Projects can experience claim failures due to other unrelated projects' activity.

Description

The cooldown mechanism tracks cumulative claims in the `CurrencyToken` account, which is derived using only the token mint (`seeds = [CURRENCY_TOKEN_TAG, token_mint]`). This creates one shared account per token (e.g., one for all USDC projects), while budgets are tracked per-project in `ProjectCurrencyBudget` accounts.

Relevant code programs/fuul-solana/src/state/currency_token.rs**:**

```rust
#[account]
pub struct CurrencyToken {
    // Global cooldown state - shared across ALL projects
    pub claim_limit_per_cooldown: u64,
    pub cumulative_claim_per_cooldown: u64,
    pub claim_cooldown_period_started: i64,
}
```

Cooldown enforcement programs/fuul-solana/src/instructions/project_budget.rs:703-711**:**

```rust
if self.currency_token.claim_cooldown_period_started + self.global_config.claim_cool_down >
        Clock::get()?.unix_timestamp {
    let new_cumulative = self.currency_token.cumulative_claim_per_cooldown
        .checked_add(claim.data.amount)?;

    if new_cumulative > self.currency_token.claim_limit_per_cooldown {
        return Err(FuulError::ClaimLimitExceeded.into());
    }

    self.currency_token.cumulative_claim_per_cooldown = new_cumulative;
}
```

Root cause: The `CurrencyToken` PDA uses `seeds = [CURRENCY_TOKEN_TAG, token_mint]` (global per token) while `ProjectCurrencyBudget` uses `seeds = [PROJECT_CURRENCY_BUDGET_TAG, project, currency_token]` (per-project). This means budget tracking is isolated per project (correct), but cooldown limits are shared globally across all projects (design flaw).

Exploitation path:

1. Admin sets USDC cooldown limit: 100,000 USDC per 24 hours
2. Project A (GameFi): Users claim 60,000 USDC over 12 hours
   - `cumulative_claim_per_cooldown = 60,000 USDC`
3. Project B (DeFi): Users claim 30,000 USDC over next 6 hours
   - `cumulative_claim_per_cooldown = 90,000 USDC`
4. Project C (NFT): User attempts to claim 15,000 USDC
   - `new_cumulative = 90,000 + 15,000 = 105,000 USDC`
   - `105,000 > 100,000` → `ClaimLimitExceeded`
5. Project C claims fail despite having sufficient budget
6. All subsequent projects blocked until cooldown resets (up to 24 hours)

Projects B and C experience claim failures due to Project A's unrelated activity.

Impact

Projects can be blocked from processing claims for up to 24 hours due to other projects' legitimate activity. One popular project can monopolize the shared limit, preventing smaller projects from serving their users. Users see unexplained "ClaimLimitExceeded" errors without indication that it's due to cross-project contention, leading to poor UX and potential reputation damage for individual projects.

Recommendation

Move cooldown state from `CurrencyToken` (global) to `ProjectCurrencyBudget` (per-project):

Step 1: Update `ProjectCurrencyBudget` struct

File: programs/fuul-solana/src/state/project_currency_budget.rs

```
 #[account]
 #[derive(InitSpace)]
 pub struct ProjectCurrencyBudget {
     pub budget: u64,
+    pub cumulative_claim_per_cooldown: u64,
+    pub claim_cooldown_period_started: i64,
 }
```

Step 2: Update `CurrencyToken` struct

File: programs/fuul-solana/src/state/currency_token.rs

```
 #[account]
 #[derive(InitSpace)]
 pub struct CurrencyToken {
     pub is_initialized: bool,
     pub token_mint: Pubkey,
     pub token_type: TokenType,
     pub is_active: bool,
     pub claim_limit_per_cooldown: u64,
-    pub cumulative_claim_per_cooldown: u64,
-    pub claim_cooldown_period_started: i64,
 }
```

Step 3: Update claim function to use project-specific cooldown

File: programs/fuul-solana/src/instructions/project_budget.rs (claim function)

```
 // Check claim limit per cooldown
 require!(claim.data.amount <= self.currency_token.claim_limit_per_cooldown, FuulError::ClaimLimitExceeded);

-if self.currency_token.claim_cooldown_period_started + self.global_config.claim_cool_down as i64 >
        Clock::get()?.unix_timestamp {
+if self.project_currency_budget.claim_cooldown_period_started + self.global_config.claim_cool_down as i64 >
        Clock::get()?.unix_timestamp {
-    let new_cumulative = self.currency_token.cumulative_claim_per_cooldown.checked_add(claim.data.amount)?;
+    let new_cumulative =
        self.project_currency_budget.cumulative_claim_per_cooldown.checked_add(claim.data.amount)?;

     if new_cumulative > self.currency_token.claim_limit_per_cooldown {
         return Err(FuulError::ClaimLimitExceeded.into());
     }

-    self.currency_token.cumulative_claim_per_cooldown = new_cumulative;
+    self.project_currency_budget.cumulative_claim_per_cooldown = new_cumulative;
 } else {
-    self.currency_token.cumulative_claim_per_cooldown = claim.data.amount;
-    self.currency_token.claim_cooldown_period_started = Clock::get()?.unix_timestamp;
+    self.project_currency_budget.cumulative_claim_per_cooldown = claim.data.amount;
+    self.project_currency_budget.claim_cooldown_period_started = Clock::get()?.unix_timestamp;
 }
```

Informational

[ILRV-2026-01.11] Missing Explicit Initialization of GlobalConfig Fields

Summary

The `create_global_config` handler does not explicitly initialize the `paused` and `project_nonce` fields, relying on Anchor's implicit zero-initialization.

Description

The handler initializes most `GlobalConfig` fields explicitly but omits `paused` and `project_nonce`, which default to `false` and `0` respectively through Anchor's zero-initialization of account data. This issue was addressed in commit `fcab55d03835ee4aedccda4adc72d90f9df7fd9c`.

Code programs/fuul-solana/src/instructions/create_global_config.rs:34-60**:**

```
pub fn create_global_config(&mut self, initial_fee_collector: Pubkey) -> Result<()> {
    self.global_config.is_initialized = true;
    self.global_config.required_signers_for_claim = 1;
    self.global_config.claim_cool_down = 86400;
    // Missing: self.global_config.paused
    // Missing: self.global_config.project_nonce
    // ... other initializations
}
```

Impact: While the zero-defaults happen to be correct, this implicit reliance reduces code clarity and makes the initialization fragile to future changes.

Recommendation

Explicitly initialize all fields for clarity:

File: programs/fuul-solana/src/instructions/create_global_config.rs (line 36)

```
 pub fn create_global_config(&mut self, initial_fee_collector: Pubkey) -> Result<()> {
     require!(initial_fee_collector != Pubkey::default(), FuulError::ZeroValueNotAllowed);
     require!(!self.global_config.is_initialized, FuulError::AlreadyInitialized);

     // general default values
     self.global_config.is_initialized = true;
+    self.global_config.paused = false;
+    self.global_config.project_nonce = 0;
     self.global_config.required_signers_for_claim = 1;
     self.global_config.claim_cool_down = 86400;

     // ... rest of initialization
 }
```

[ILRV-2026-01.12] Inefficient Early Return Logic in Fee Update Function

Summary

The `update_project_fees` function uses early returns when individual parameters haven't changed, causing the entire transaction to fail even when other valid updates are provided. This issue was addressed in commit `6b778b2604f722ca555957d1a4e02942d424d1e1`.

Description

The function checks each fee parameter individually and returns an error immediately if that specific parameter hasn't changed, preventing other valid updates from being applied.

Code programs/fuul-solana/src/instructions/project.rs:76-136**:**

```
pub fn update_project_fees(...) -> Result<()> {
    if let Some(user_native_claim_fee) = user_native_claim_fee {
        if user_native_claim_fee == self.project.get_user_native_claim_fee(&self.global_config) {
            return Err(FuulError::NoNewChanges.into()); // Early return
        }
        self.project.fee_management.user_native_claim_fee = Some(user_native_claim_fee);
    }

    if let Some(project_claim_fee) = project_claim_fee {
        if project_claim_fee == self.project.get_project_claim_fee(&self.global_config) {
            return Err(FuulError::NoNewChanges.into()); // Early return
        }
        // ...
    }
    // ...
}
```

Impact: Users attempting to update multiple fees simultaneously will have their transaction fail if any single parameter hasn't changed, wasting transaction fees and creating poor UX.

Recommendation

Use a flag-based approach that only fails if no changes occurred after processing all parameters:

File: programs/fuul-solana/src/instructions/project.rs (update_project_fees function)

```
 pub fn update_project_fees(
     &mut self,
     project_nonce: u64,
     user_native_claim_fee: Option<u64>,
     project_claim_fee: Option<u16>,
     remove_fee: Option<u16>,
 ) -> Result<()> {
+    let mut has_changes = false;

     if let Some(user_native_claim_fee) = user_native_claim_fee {
-        if user_native_claim_fee == self.project.get_user_native_claim_fee(&self.global_config) {
-            return Err(FuulError::NoNewChanges.into());
+        if user_native_claim_fee != self.project.get_user_native_claim_fee(&self.global_config) {
+            self.project.fee_management.user_native_claim_fee = Some(user_native_claim_fee);
+            has_changes = true;
         }
-        self.project.fee_management.user_native_claim_fee = Some(user_native_claim_fee);
     }

     if let Some(project_claim_fee) = project_claim_fee {
-        if project_claim_fee == self.project.get_project_claim_fee(&self.global_config) {
-            return Err(FuulError::NoNewChanges.into());
-        }
         if project_claim_fee > BASIS_POINTS {
             return Err(FuulError::InvalidPercentage.into());
         }
-        self.project.fee_management.project_claim_fee = Some(project_claim_fee);
+        if project_claim_fee != self.project.get_project_claim_fee(&self.global_config) {
+            self.project.fee_management.project_claim_fee = Some(project_claim_fee);
+            has_changes = true;
+        }
     }

     if let Some(remove_fee) = remove_fee {
-        if remove_fee == self.project.get_remove_fee(&self.global_config) {
-            return Err(FuulError::NoNewChanges.into());
-        }
         if remove_fee > BASIS_POINTS {
             return Err(FuulError::InvalidPercentage.into());
         }
-        self.project.fee_management.remove_fee = Some(remove_fee);
+        if remove_fee != self.project.get_remove_fee(&self.global_config) {
+            self.project.fee_management.remove_fee = Some(remove_fee);
+            has_changes = true;
+        }
     }
+
+    // Only fail if no changes occurred after processing all parameters
+    require!(has_changes, FuulError::NoNewChanges);

    emit!(LogProjectFeesUpdatedEvent {
        project: self.project.key(),
        user_native_claim_fee,
        project_claim_fee,
        remove_fee,
    });

    Ok(())
 }
```

[ILRV-2026-01.13] Missing Role Existence Check in revoke_project_role

Summary

The `revoke_project_role` instruction handler fails to verify that the target account actually possesses the role being revoked before attempting to remove it. This allows successful execution of role revocation transactions even when the account never had the specified role, resulting in misleading event emissions, wasted transaction fees, and potential confusion for off-chain systems monitoring role changes. This issue was addressed in commit `c675f80e881f51a606d6c0e6d7ea61b31810c13e`.

Description

The function documentation explicitly states "The account must have the role" as a requirement, but this validation is not enforced in the implementation. The underlying `revoke_role` method uses `retain()` which succeeds silently regardless of whether any role was actually removed.

Relevant code programs/fuul-solana/src/instructions/project_roles.rs:77-94**:**

```
/// Revoke a role
///
/// Requirements:
///
/// - The account must have the role.  // Documented but not enforced
/// - Only admins can call this function.
impl<'info> RevokeProjectRole<'info> {
    #[allow(unused_variables)]
    pub fn revoke_project_role(&mut self, project_nonce: u64, account: Pubkey, role: ProjectRole) -> Result<()> {
        require!(account != self.authority.key(), FuulError::CannotRevokeSelf);

        // No check: Does the account actually have this role?
        self.project.roles_mapping.revoke_role(account, role.clone())?;

        // Event emitted even if nothing was revoked
        emit!(LogProjectRoleRevokedEvent { account, role });

        Ok(())
    }
}
```

Underlying implementation state/project.rs:112-115**:**

```
pub fn revoke_role(&mut self, account: Pubkey, role: ProjectRole) -> Result<()> {
    self.roles.retain(|r| r.account != account || r.role != role);
    Ok(())  // Always returns Ok, even if no role was found
}
```

Root cause: The `retain()` method filters the roles vector but doesn't report whether any elements were actually removed. The function always returns `Ok(())` regardless of whether the role existed.

Impact

Transactions succeed and emit events even when no role was actually revoked, leading to: - Misleading event emissions that don't reflect actual state changes - Wasted transaction fees for operations that have no effect - Potential confusion for off-chain systems monitoring role changes - Inconsistency with documented requirements and `grant_role` validation patterns

Recommendation

Update the `revoke_role` implementation to check for role existence before removal:

File: programs/fuul-solana/src/state/project.rs (lines 112-115)

```
 pub fn revoke_role(&mut self, account: Pubkey, role: ProjectRole) -> Result<()> {
+    // Verify the account has the role before attempting to revoke
+    require!(
+        self.has_role(account, role.clone()),
+        FuulError::RoleDoesNotExist
+    );
+
     self.roles.retain(|r| r.account != account || r.role != role);
     Ok(())
 }
```

Add the corresponding error variant:

File: programs/fuul-solana/src/errors.rs

```
 #[error_code]
 pub enum FuulError {
     // ...
+    #[msg("Account does not have the specified role")]
+    RoleDoesNotExist,
 }
```

This ensures that: - Only valid role revocations succeed - Events accurately reflect actual state changes - The implementation matches the documented requirements - Behavior is consistent with `grant_role` validation patterns

[ILRV-2026-01.14] Missing Global Role Check in renounce_global_role Instruction

Summary

The `renounce_global_role` instruction handler fails to verify that the caller actually possesses the role they are attempting to renounce. This allows successful execution even when the account never had the specified role, resulting in misleading event emissions and wasted transaction fees.

Description

Similar to I-03, this function allows users to "renounce" roles they don't actually have. The underlying `revoke_role` implementation succeeds silently regardless of whether the role existed.

Relevant code programs/fuul-solana/src/instructions/admin_roles.rs**:**

```
pub fn renounce_global_role(&mut self, role: GlobalRole) -> Result<()> {
    // Missing: Check if authority actually has this role

    require!(
        role != GlobalRole::Admin || global_config.roles_mapping.count_roles(GlobalRole::Admin) != 1,
        FuulError::CannotRenounceLastAdmin
    );

    global_config.roles_mapping.revoke_role(self.authority.key(), role.clone())?;
    emit!(LogGlobalRoleRenouncedEvent { ... });
    Ok(())
}
```

Root cause: No validation that the caller possesses the role before attempting to renounce it.

Impact

Identical to I-03: - Misleading event emissions - Wasted transaction fees - Confusion for off-chain monitoring systems - Inconsistency with expected behavior

Recommendation

Add role existence validation before allowing renunciation:

File: programs/fuul-solana/src/instructions/admin_roles.rs:

```
 pub fn renounce_global_role(&mut self, role: GlobalRole) -> Result<()> {
+    // Verify the authority has the role before renouncing
+    require!(
+        self.global_config.roles_mapping.has_role(self.authority.key(), role.clone()),
+        FuulError::Unauthorized
+    );
+
     require!(
         role != GlobalRole::Admin || self.global_config.roles_mapping.count_roles(GlobalRole::Admin) != 1,
         FuulError::CannotRenounceLastAdmin
     );

     self.global_config.roles_mapping.revoke_role(self.authority.key(), role.clone())?;
     emit!(LogGlobalRoleRenouncedEvent { ... });
     Ok(())
 }
```

[ILRV-2026-01.15] Integer Division in Fee Calculation Causes Precision Loss and Revenue Leakage

Summary

The `remove_fungible_token` function uses integer division to calculate removal fees, which causes precision loss for small withdrawal amounts. This results in zero fees being charged when fees should be collected, leading to protocol revenue leakage. This issue was addressed in commit `73b2ef801dfe2fe5ca7c1635657ae53577442fe2`.

Description

The `remove_fungible_token` function calculates removal fees using integer division, which truncates any fractional part. For small withdrawal amounts, this results in zero fees being charged when a fee should be collected, leading to protocol revenue leakage.

Relevant code programs/fuul-solana/src/instructions/project_budget.rs:574-579**:**

```
// compute the remove fee
let remove_fee: u64 = project.get_remove_fee(global_config).into();
let fee = remove_fee.checked_mul(amount)
    .ok_or(FuulError::Overflow)?
    .checked_div(BASIS_POINTS.into())  // Integer division truncates fractional fees
    .ok_or(FuulError::Overflow)?;
let amount_after_fee = amount.checked_sub(fee).ok_or(FuulError::Underflow)?;
```

Constants src/constants.rs:32**:**

```
pub const BASIS_POINTS: u16 = 10000;
```

Root cause: The fee calculation formula `fee = (remove_fee × amount) ÷ BASIS_POINTS` uses integer division in Rust, which truncates (rounds down) to the nearest integer, losing any fractional part.

Example with 2.5% fee (250 basis points):

```
amount = 39
remove_fee = 250
BASIS_POINTS = 10000

fee = (250 × 39) ÷ 10000  // 9,750 ÷ 10000 = 0.975
fee = 0                   // Truncated to 0 (should be ~1)

// Threshold: amount = 40 → fee = 1
// Just below next threshold: amount = 79 → fee = 1 (should be ~2)
```

Token types supported by Fuul: - **Fungible tokens:** 6 decimals (e.g., USDC) - **Non-fungible tokens:** 0 decimals (NFTs) - **Native SOL:** 9 decimals

Impact

For 6-decimal tokens (USDC): - Minimum amount to pay fee: 40 base units (0.000040 USDC) with 2.5% fee - Impact: **Negligible** - affects only dust amounts - Revenue loss: < $0.001 per 1000 small withdrawals

For 9-decimal tokens (SOL): - Minimum amount to pay fee: 40 lamports (0.000000040 SOL) with 2.5% fee - Impact: **Negligible** - affects only dust amounts - Economic impact: essentially zero

For 0-decimal tokens (if used as fungible): - Minimum amount to pay fee: 40 whole tokens with 2.5% fee - Impact: **Low-Medium** - withdrawals of 1-39 tokens pay zero fee - Could be significant for high-value tokens (e.g., $10 token × 39 = $9.75 lost per transaction) - Note: 0-decimal tokens are primarily used for NFTs, which use `remove_non_fungible_token` (no fees)

Recommendation

Option 1: Enforce Minimum Withdrawal Amount (Recommended)

Prevent withdrawals that would result in zero fees when a fee is configured.

File: programs/fuul-solana/src/instructions/project_budget.rs

```
 pub fn remove_fungible_token(&mut self, project_nonce: u64, amount: u64) -> Result<()> {
     let global_config = &self.global_config;
     let project = &self.project;

     // Validate inputs
     require!(amount > 0, FuulError::ZeroValueNotAllowed);

     // compute the remove fee
     let remove_fee: u64 = project.get_remove_fee(global_config).into();
     let fee = remove_fee.checked_mul(amount)
         .ok_or(FuulError::Overflow)?
         .checked_div(BASIS_POINTS.into())
         .ok_or(FuulError::Overflow)?;
+
+    // Enforce minimum fee if remove_fee is configured
+    if remove_fee > 0 && fee == 0 {
+        return Err(FuulError::AmountTooSmall.into());
+    }
+
     let amount_after_fee = amount.checked_sub(fee).ok_or(FuulError::Underflow)?;
```

Add new error:

File: programs/fuul-solana/src/errors.rs

```
 #[error_code]
 pub enum FuulError {
     // ...
+    #[msg("Amount too small to cover minimum fee")]
+    AmountTooSmall,
 }
```

Benefits: Prevents zero-fee withdrawals, simple implementation, clear error messaging

Drawbacks: Blocks legitimate dust withdrawals

Option 2: Round Up Fee Calculation (Alternative)

Always charge a minimum fee of 1 unit when a fee is configured.

File: programs/fuul-solana/src/instructions/project_budget.rs

```
 let fee = remove_fee.checked_mul(amount)
     .ok_or(FuulError::Overflow)?
     .checked_div(BASIS_POINTS.into())
     .ok_or(FuulError::Overflow)?;
+
+// Charge minimum 1 unit if fee would be 0 but remove_fee is configured
+let fee = if remove_fee > 0 && fee == 0 {
+    1
+} else {
+    fee
+};
```

Benefits: Allows small withdrawals while always collecting fees

Drawbacks: May charge disproportionately high percentage for very small amounts

Option 3: Document Behavior (Minimal)

Document the minimum withdrawal thresholds without code changes.

Benefits: No implementation required, provides transparency

Drawbacks: Does not prevent revenue leakage

Appendix: Proof of Concept Exploits

This appendix contains a base64-encoded ZIP archive with working Proof of Concept (PoC) exploits for the two critical vulnerabilities identified in this audit. The archive includes complete test files that demonstrate the vulnerabilities and can be integrated into the project's test suite.

Archive Contents

The ZIP archive contains the following PoC files:

1. `c01-multisig-bypass.test.ts` - Proof of Concept for ILRV-2026-01.01: Ed25519 Multi-Signature Bypass
2. `c02-replay-attack.test.ts` - Proof of Concept for ILRV-2026-01.02: Proof Parameter Substitution Enables Replay Attacks and Attribution Fraud

PoC Descriptions

ILRV-2026-01.01: Ed25519 Multi-Signature Bypass (`c01-multisig-bypass.test.ts`)

This PoC demonstrates how an attacker can bypass the multi-signature verification mechanism by combining signatures from authorized signers who signed different messages. The test shows how one legitimate approval can be combined with signatures from unrelated claims to satisfy the multi-signature threshold requirement, allowing unauthorized fund transfers without proper multi-party review.

Important: This PoC requires a patch to `@solana/web3.js` to simulate SIMD-0296. In the library file, change:

```
const PACKET_DATA_SIZE = 1280 - 40 - 8; // 1232 bytes
```

To:

```
const PACKET_DATA_SIZE = 4096;
```

This modification simulates the increased transaction size limits proposed in `SIMD-0296` , which is necessary to demonstrate the multi-signature bypass vulnerability.

Expected Output:

When the test runs successfully, you should see output similar to:

```
[C-01] Ed25519 Multi-Signature Bypass Vulnerability

Budget before: 1000000000 lamports
Budget after:  999999000 lamports
Drained:       1000 lamports
Attribution:   1000 lamports

✓ ATTACK SUCCESSFUL: Multi-sig bypassed using signatures from different messages
  ✓ PoC: Should succeed bypassing multi-sig by using signatures from different messages
  ✓ Control Test: Proper multi-sig works when all signers approve same message
  ✓ Control Test: Should FAIL when only 1 signer approves (need 2)

3 passing (243ms)
```

This demonstrates that the attack successfully drained funds by combining signatures from different messages, bypassing the multi-signature requirement.

ILRV-2026-01.02: Proof Parameter Substitution Replay Attack (`c02-replay-attack.test.ts`)

This PoC demonstrates how a single signed authorization can be reused unlimited times with different proof values to drain the entire project budget. The test shows how the lack of validation between instruction parameters and signed message fields enables replay attacks and attribution fraud, allowing systematic fund theft and corruption of attribution data.

Expected Output:

When the test runs successfully, you should see output similar to:

```
[C-02] Proof Parameter Substitution Replay Attack

Claim #1:
  - Using fake affiliate: 0xbbbbbbbbbbbbbbbb...
  - Computed proof: 0xabd9c08934c9d63c...
  - Creates new PDA: E6Y732Ft51CYoFcY...
  SDK returned 2 instructions
  Replaced Ed25519 instruction with ORIGINAL (replay attack)
  Transaction logs:
    Program 7BLKqULKe2eMmrjqMZSaX8iGsa5tue9LqZTQuEpTBEg invoke [1]
    Program log: Instruction: Claim
    Program 11111111111111111111111111111111 invoke [2]
    Program 11111111111111111111111111111111 success
    Program 11111111111111111111111111111111 invoke [2]
    Program 11111111111111111111111111111111 success
    Program 7BLKqULKe2eMmrjqMZSaX8iGsa5tue9LqZTQuEpTBEg consumed 50052 of 203000 compute units
    Program 7BLKqULKe2eMmrjqMZSaX8iGsa5tue9LqZTQuEpTBEg success
  Transaction result: SUCCESS
 ✓ Claim #1 SUCCEEDED - budget now: 999999900, drained: 100

Claim #2:
  - Using fake affiliate: 0xcccccccccccccccc...
  - Computed proof: 0xcbc925609a9b58a7...
  - Creates new PDA: AtEqP9pzttg8LeS1...
  SDK returned 2 instructions
  Replaced Ed25519 instruction with ORIGINAL (replay attack)
  Transaction logs:
    Program 7BLKqULKe2eMmrjqMZSaX8iGsa5tue9LqZTQuEpTBEg invoke [1]
    Program log: Instruction: Claim
    Program 11111111111111111111111111111111 invoke [2]
    Program 11111111111111111111111111111111 success
    Program data: 568azFHIVwATrM1cBbsTr4W4LmwGdiiFP44Z7PxCpj/Cx0jSRDXJLmQAAAAAAAA
    Program 7BLKqULKe2eMmrjqMZSaX8iGsa5tue9LqZTQuEpTBEg consumed 48786 of 203000 compute units
    Program 7BLKqULKe2eMmrjqMZSaX8iGsa5tue9LqZTQuEpTBEg success
  Transaction result: SUCCESS
 ✓ Claim #2 SUCCEEDED - budget now: 999999800, drained: 100
```

This demonstrates that the same Ed25519 signature was successfully replayed twice with different proof values, draining funds twice from a single authorization.

Base64-Encoded ZIP Archive

Archive Name: `Fuul-PoCs.zip`


```
UEsDBBQAAAAAAJVgnVsAAAAAAAAAAAAAAAKACAARnV1bC1Qb0NzL1VUDQAHSmB5aUpgUmlKYFJpdXgLAAEE9QEAAAQUAAAAUEsDBBQACAAIAIJgnVsAAAAAAAAAADIyAAAlACAARnV1bC1Qb0NzL2MwMS1tdWx0aXNpZv1eXBhc3MudGVz
dC58c1VUDQAHJGBSaUxgUmlKYFJpdXgLAAEE9QEAAAQUAAAA7VvpchvHEf7PpxhXpcKFBC4PSS5nTGQVBiSTyKgJS4YBY92B0AY+7l3VmSsKynyN88XZ4kPef07AIkdDj5EaFYFjBHT/fXx3T3rrefPNlAT9DF5fnSMVK//vlZf3AxClB/
a2cXDaK9Fy92/4JOo5jRr5GdpZhVBUGHixyXJXpXx5kpBITGlC2ADKc0mtM5MVIvFJEk50tWYP1F5tkdwinCjOHwhQohB8T5SURxEtDvCC/VrQzCUlZv5lOFiiMKglNaTpDZlmJpkWWolhOp65ApSghZYlnME5TlIEMK9NZTIwIlLN5hYxm
qa9YdqFmEHOC8iKDrThBwFcFBBGDY/tlmnIhpBEt65gU44BYkCmxAnRDHXRpGIwCrDAX5ovRNJpVoQk8sWRmlpeEBAzpBDzLY5pJGAiQJxzoUGeIbSuYivEmV3qT4FmlITuA/KYsXWsKTJMchCIcyeTvSJ2HFgBgKY0wTCXGVMo8wKUp0
NwcUcwDklkQ1zJyq2FN2uVbymDASL5Qq+XYuflOjJzxWAMYoySISc+a2NzZokmcFQ+/RawKbadFFF9UkpiH87KJRgdMSC2hPapSdcfRBWsDmD2UW4xRv35HJM/+XcnPfkN5AKM5wdFylMzqJSb8qQIRwMcpuSHpM7zlv3caaI5JnJWWN2Zdx
NsFxP0undNaYuiivX0iY3DC8TYbRjUU8IlEvFBh3NzTjvr9dMRqX21O52WYdvaGMDN+dGinBRkl5m7jS9bkmTpWdNX4fYYbN2CXBJeAHP4+rKlasSakusiJs1duyBAyMDwiYRouc1Bz/cIdvSJXHeAJMAGEtCb3Ndwk2aKmsRC1pXSnAFnPA
zywL55jWQ2GxyMGbjZT4UU8+AtcFfwznWah5C7MCxIv3i9+25Tgs3ohIGR2QrzMQ9qVx8VITa7vOugz++lMRGGgPtAK2dfj0U3qQZWI80EuL0ooWmgTVzPTZWxCYRPIV4Ft e3FMQHozhisGAotI1jgOokIWqPSz3dXiO9Z
4zAxIRCyyACiKYfLRRqaoCDkveeodDkMoMYDhQ8wZZYbeJ3OvI5owdJYu8THHMIWESV+1wKqQcz1yhYQQSjktxMY4BJtMP2ii4BRBIHTJf3QRPEkKEgcqb5Kh+MfCwA59EHJIWIDTjR8KJbhLjx6pGzNW7qpxRN7ZqdrydwsNWgH23xKfm
2kGRM3WGGb1dHmateZdlC1np1p46dkl4FDMKrhqK1aadaeu+6lreCSBNNa7vCrhbc+FvTMupQ78F2fRL3wUdXiqXb26hfEMvy5zTiaon0nSymld+udarv5bXPfQnXK0NDsQsVcDGABaJJxuYOB5JWM9B7A1vbvNFYqdL3fU9tim78GT+kvnuM
zSI7hFl0/NzkBfVKyXg0Vi2hvMfWJecL+XSCyAQdP9b/OI9Dn9xlOKGEKBVfmysgutoD0FBrvOuh9TUihCtkGT2AhpViNROzh/EiCaa5ieZwVI8J0F4tb7f2b+VlLHmbF1k/QMN5VsURGr7t9weDIyvdNCLzHLYq1vsR4Fjv321c3dHg
AsGN2BPOGgonLE2SzBHe7UIERDEWkUogPwdpSDy1YqFwdSEd+F1K7pysZlAHqMLN3pzm5zuNrwq9h3R2As66ew2nd2qQTRpQJ4M24iHNKeHbBUfLtMJ0ihfU2Z4vQJFjUSNL8SMyxQC9w8s2/TsFe6jYheZLZmw+OF5UJYcLQNN7tekeeRp
aWDnqH5vOoXMC5C/wAugITQPQE5kqBi7JZmigFN13EkUSJcmmCc+4nV8M1fzAdVbSTkju/VYRHAU05QY9E8xm/vTOMsKD9RE/DS7A/vZFvd1Bz1Fz761KP21TeNZRrDjjAPLGPxud16YxXQHKfnZgkJ2WE2sYzy6DT4eHA5QBtZSvlip2Wf
wpYtFibZ5A+xzRc70jg5w0fG4+Sibz7bYgUMvKg2a7hsN4TkSnp6jyivfWmI4+1ebsQiJ3/txYP5ubYuzY/x9r3Wtb+DILwaNTrv0ZbqC+ap2SNuG6dm3AlollVqmLv5B4Ol85z2pyqb0d9OY4NJu/ry4Om+453rnwzZ1TkXBUBH3oLqfn
gFMgtb1lLK7Il7EVRVh2WHFcPLBDA3Y9eGVdsgLugzU/jnqeB2igW0iir/RjlJSFVBEY66EoME2iWpsw9zSd2x+KpZ6KHWCOaUi6ba91DJCmlFEcy63cAK3j/ubLX4DPWZVMOD7o99/RjpUQEyY6WirI1JZinRBV5bJYO6jX+wRNp1ViSy9m
jX9biZb5Zr7QFTBnHBcTYhVMuuTrhJYPJpC6YgQNGVaWvNJ/oVfNSUZt0b8mKQzqCO2OO7V/jI/HXFC47bbdvU5V42CO2oFOn1B3QSD72SI15HbNGxUm632oaOj+gY89ghcUoY5hfkEIkGR+fjpDu08sjtc2qUnOMFKUCdD+oYLkAIeYdx
Ft7McTkPeCLvxzwnqAc9E5Qtiw8tWmBrjfqo5zzK3uloflzZbw2M90qEZcB65kqFnrVjuYAR6colz1N3Y/cqCq9XbLr28aLdt9D76RR5fbeXEYOJZ9PVyuoqDbI8lRQFpESHqnvo9BQ/Az7o5xSekX7YIAz3J5xNitNCQ6sK723IDIU2Z2k
pWflEEdJUPM2beODcVYCtAl3Jz+AZUNWQMliqH/Y+Kyw/CLgugHThYrKqK2CkBAo9z87PvemvAv25cPzlKat4C2OWh2brchbQA4EhfBBI8pv2WQbYaNmu8cA/EnFNeM2EE1kWNHtdgQYY4fMFVBZdYQpvR/Nz9rZsby/fZ74eNb1wm3bFxlm
XT3b9UZ9/hLdqdNVr8LPWzz7U8J4x9eRwkrVeg0rU43T0iiGN7BsHmQCFhkOedcD3Mv7+adUqU42mgP0p/eOSj+YmuZnC32zXW3G3HACBJst/T+y9UiyGkiYYavi/ZFtFkqB2GaJ6cuqx7Jcl5igJh97eOq0Diz2ya8Vjlwrv28vepD2iq02
tSs/pf/+1z91R1zaKMPh8ds3gXoEItsnvKUCOlu7idJs1PS2lBVZjEZwFUEtVWQSsVpS6C4rbvhTRZKap6q8eaUeMTqPTJf2Z742V74ZV5aUmpZt5MLkIsrEdX3FK08evQS1i9uxQ0l8jQGqSW7a/VM3i183k18rl38wm3fz+SY0jzVdXV9W
3df3sKb6cBZGi/QrsJQu3CJvB5uAbTX+eq/X/33E/yX4LOG5g577IigNjxJjRULk+6vcl7MfZc4BEMuvL4Tr+nGjzy7crlaK1zU5rldMHnZYa3qZx2juFylYUHugMLRGcZG6RZNZu/428ZqecxOqf9gEIMK2+qcMT7BUtLp9OSV23iqG/r
W8jO5UBd1riMeXqrSFloGsZVRLzNhxlScUmYyrZ8Wa1veuCmoF36TpYQVyQgeey2D6H2q9vndfdQvbzDUPfbVe+hKvOE2rNLTS3kf6jna4rdzbyG6fNcrCrzYN13Bi7obOBYtPS1EoCoFPtoJVrxHJTQqtTk8
eXn9ZnAGyvz2+b4ZvXh7+HrwoSp4tldPvBr0jgaXasIa5ZQAy0u9Z/e5vEpkia7k0j2gbw4ODL9qTBtZq3yWWS6C2t4CPaIE54gbamrmG3nphuEtoekcFCfekeEN4FWSJBBS1mXviyiLE4E/kcRxWvMOCR01Lwt9MFG/LyF9Ai5Fb+q5tjKl
Rck2WtHuXBIMlCxjXkKORZOM26nHT6EwtLMP//yIzTEMP32q8WsShaBUzjJpPIoxfHTA617AxdwPNLpuQxm6IYsSsOxCXBfkuMm0QC5ruZyxTR/U9K02kqSY/7eKdwR3ujVyRDB3+jVAHFm4/j9QWej0aL7OLkaJux15RJ8UCZNZkyvNCVL
ts0KQhC3oxrPjQzjKlFyZ2U7pTVuyGRKctD5hLxLRfnvLNYREzmOIojjimD9ohzkTQUFg96SqOcZRCjxsBG3w+onGQWeCZhZcS6jcPOFcui3T4AFFCbAjtVr+Z7RzP51zmgusFd9/j77+1OvazFiauOavN5LA7s+dQeX74YPp0G+aVnNg6Q
XnAN7tYW78xOiqdZYm0ZvUqYc2CFbJarPxBwirLuQYLiHgSrXlHJ9foKGv/4oMqLPe6Q3lwEjkWviu16ZBIx68So06hUr4HCYLeivSKMYHfINq3v6waiY+wCfYv4Rlr+wxuo4IMvSy+WN7Ef7dt3jd+Sp7c01Wd/3RSIj+5i8BWSWd1Yd
S95EX+I8wlNUWXB1Vh5r/ncCT+j3dDAc9l4O0HBw0bvsjQZvfuy43Oirx8pEHuNHPzKpWSgICJI2nxPZiaK6NOsKjq812aC3qXJLiNXyScDuGs+mkiA4sgEaqyfD8kax2831+UlX5+Eb/wFQSwcIACRfh+8MAAAyMgAAUEsDBBQACAAIAIJg
nVsAAAAAAAAAAAE0BAAAwACAAX19NQUNPU1gvRnV1bC1Qb0NzLy5fYzAxLW11bHRpc2lnLWJScGFzcy50ZXN0LnRzVVQNAAckYFJpTGBSaWZgUml1eAsAAQT1AQAABFAAAABjYBVj2ZB1YPBNTFbwD1aIUIACkBgDJxAbMTAwSgNpIJ/R14Eo
4BgSEgRhgXWcAOJWNCVMUHEPBgbV5Pxcvc5CgpxUvdzUksSUxJjEqZxfF8+5INzwiNSiVLei/Nx1kFkCQMKWgEUOb6wNLEoMa8kMy+VIakgJ7O4xMBzIWO8gGRGSUlBsZW+fnlqkl5JakSqelFirl5+Ubo+BxfUXYx7kAGaoX6BgYWhtZm
lqZGZgZGZtZhmWWIOSmZlqaWZkYGTlaGuiauFm66JoZmlrqWRkYWum6GZgaWzmVupoam5gwAUESHCJ8rvyjIAAAATQEAAFBLAwQUAAgACACDYJ1DAAAAAAAAAACUMQAAiwAgAEZ1dWwtUG9Dcy9jMDItcmVwbGF5LWF0dGFjay58ZXN0LnRz
VVQNAAcmYFJpTGBSaUpgUml1eAsAAQT1AQAABBQAAADtG9tuG8f13V8xQY1wGVOrixsDpeu1FEUahC2JEGkHhSLYw92hONFyl9mLJUbgV/ShL/m6fEnPmdvO7C4ppkHQCyokCHcuZ879NpPDb755Rr4h46vLyyGBf/qXF/3BeNol/YOjEzJO
k2ROx1S1SSazlEyKWZbzvMh5EpMrtoromvTynAZ3AhTBc8IznLchKvZRJneUrhiyvSe0JiQsVSgBPAx2v9ollGUgllISY5CwTc2ZpK5pz4FsBSdMYBVCrWAUZfaFQA2HxB4SA+nwPIeZosyT0O3NOMOCJfjCn/iYWEx7COkYzfxvCi2FIG
b5mv8P1YRDFLGYxHPF93AX1Ggojypdcm8yKW+MBhPBRkiPMkBivNk4xkNOfZfC1OWdBsgXBTFlHcnC34inhyyxtyx4KA3snPT/ccUCzyY/DiA1BOXhDiqS0msyInF4OPg6vycIQqzodjMmaFv6RSsGigkcw5i8JMkzparmiQd/Hnse8Kr0t6
sDe+jQwIzT9BhBBXmFLFSRbnPGUaW0A1vGu5gj3xEV7KZlI/hiktwi4ZUMBOMJUEXQt5nlnipPM5sF4wl4O2MBoS4BSeYkkwiRmAP3z2jC9XSZqTR/K0gFlwtEPGxSziAXy5jd5AIl+zJKIvPbxnsSf+DlnrtdnIjApoeGWiG/5LGL9jKlZ
HKynyR2Ln/whL1LWqaw5YGsk43ll9m2UzGiUT+15v61MiSVTKqOTLB1tzJeGcBD1nYC4Kk1PPOM4247x8C36LscC4326iT9zxnk4/nhkpQV5Z9WbrU9ZHJ51L4ncr3Gc2pGbtiNEti/BwWRaRQk1RdJZHYq7clSxA7Dgg2TdcrMQsCV5RaAh+H
1J0TA0Yu9/SOFaB0M6AQTj2QcrKJzFgcjtBnFMLwMlJjjUsyYQ8r1EC9LFhQXk4H6XqVJ3pKfJST4FJADeMAVEzjFyQpjQ4e1j8dynH7IGm4n06+fVXqWZyAghyiwTNAbEffwoZnIcsC4AfzWtfoQG9+hQdtdQh4nTd/kYrKcgLEdrXgX+ux
EIxVCU2P3QrB9cIlj7vaMvTcykkiW9pKZbILjDBjyuzQGVZApSzgKw7GX5sRPiO1hmF8xuZJytD2PZqt48CijRDvEYnrIDXAzzeE3lOeW5bitduv9UKLusraBjN0AFtAFKUdix8VYK7hluAIQIeHpBeG5AKc4hdGcjQIiFQr1BExH2DAA2lB
eHJRo4+x5l3/YqEkNc8TYEmfbxAzisC5iu2nWhR1XJ5uAeK1mzXnn9iMaafMvD1MvRT6dovCVcr1MNuxwd6AFphYZRsr9V5BZ5cqnf8m1IAPJGbUnBDFuFV3+MJciIGdci1kobv+57aFN75twi09JJeo1pFbFQy4PzrbYadcqVEt6s1glqC
6HYtH+1LMvQCUFL580awRkmG51SrnPS7ZAKZRhSSyYd+fzA4UzkKBHzIXZYy5lOhgMdHJOeQPhBMT1RmhcIQGUsGHqpuy8DXyXQwJsddcorJXRximBcRncJvhJ2Ry4uBm1JY9hJYUQpEGbN7J3AZhkIWBEGsOo1xzeb5UjLylN+OQP7HR0ft
kofG362s+GQYXLo387NJELmOg90yJPrSZCqLzjkCM17XD9mcFlHu4JPMv5Np4FjjdlogO30MMl6r1/JBVsBN79Uf2x3SWrCHVtvGGeN41w7qfk9nVGO6Brilguhfo0QjpkpNOlLfYCJiBXi9HkL4JHRchHUe8tm/mSjy+QBrKE2HjciyEXC7i
MTQvOKf5wp9HSZJ6IDTmx8k96NEh6BwICbLfl68sYWl0jYuRumI0NQNNNsRXHxxnVWtrPiXIPCg/o3C2PaTQIT7799vhPo4fqVqmuerr0AS1ZcxAYX0VMWht4BYOA5VeE/p90yYcM/YqwrWYLUnWLhRyPec5pdCpvaEDMdiwmi9L+XK7SQUEp
MfAxDozqbdG2dvsrXhpfo5cfMsZs1NgIjg8RAAN+ZFJfQJuXtk8IAk/ywcs1TMg/IIGNc3sPD65rWBs0rZf54UmaHHoc8RpcnRRYaEJJr/2jplR44fJrMWlm5R5/s1xb2fzSNOKstvlFnQIJCPMSUCxLhP38mJ/CFfy/aRKs/4ocoNkpu
vc8qYyZ/eP7IQUmPN93PmnF23DsbDYeDq8HFVAkaz2Gm8LEA51CugYSQSeOG+QNStv1XeHTNb7QPeL0DZAWIscgyiYXUPOwOgHnXu1CycyZQHrnTa9+0ja2W/KhwjpADsAkMM3PI/UuJdMnRw/PHXYfCQROoK+JbT5Lri4aAGDjqkONX7Q0E
4c80Fvrn9pPlqsiheBTyaDpx7zNKYZ+xFDO/8VlPCDjHZkcR8x8LJo9xhFJyIa2R0Ga21U+lsPZxu01hzKVxLxH1Vbj GgACkdYnFKRe9kmX7cGsoWMRU1GdD7hkcARKBomyZgEeUHRPZGdAtC5grMrbLbwrQpx+moM5sJbsemMMo9231lMt79
goMBygQE1+LA4MBsOzKTqO6RjQyZjpQJiRNxcK8vZzrWWpOfCEloT+7Vt+pGkb3Zyl+e2m6WOgD2SmyuzQGVZApSzgKw7GX5sRPiO1hmF8xuZJytD2PZqt48CijRDvEYnrIDXAzzeE3lOeW5bitduv9UKLusraBjN0AFtAFKUdix8VYK7hluAIQIeHpBeG5AKc4hdGcjQIiFQr1BExH2DAA2lB
eHJRo4+x5l3/YqEkNc8TYEmfbxAzisC5iu2nWhR1XJ5uAeK1mzXnn9iMaafMvD1MvRT6dovCVcr1MNuxwd6AFphYZRsr9V5BZ5cqnf8m1IAPJGbUnBDFuFV3+MJciIGdci1kobv+57aFN75twi09JJeo1pFbFQy4PzrbYadcqVEt6s1glqC
6HYtH+1LMvQCUFL580awRkmG51SrnPS7ZAKZRhSSyYd+fzA4UzkKBHzIXZYy5lOhgMdHJOeQPhBMT1RmhcIQGUsGHqpuy8DXyXQwJsddcorJXRximBcRncJvhJ2Ry4uBm1JY9hJYUQpEGbN7J3AZhkIWBEGsOo1xzeb5UjLylN+OQP7HR0ft
kofG362s+GQYXLo387NJELmOg90yJPrSZCqLzjkCM17XD9mcFlHu4JPMv5Np4FjjdlogO30MMl6r1/JBVsBN79Uf2x3SWrCHVtvGGeN41w7qfk9nVGO6Brilguhfo0QjpkpNOlLfYCJiBXi9HkL4JHRchHUe8tm/mSjy+QBrKE2HjciyEXC7i
MTQvOKf5wp9HSZJ6IDTmx8k96NEh6BwICbLfl68sYWl0jYuRumI0NQNNNsRXHxxnVWtrPiXIPCg/o3C2PaTQIT7799vhPo4fqVqmuerr0AS1ZcxAYX0VMWht4BYOA5VeE/p90yYcM/YqwrWYLUnWLhRyPec5pdCpvaEDMdiwmi9L+XK7SQUEp
MfAxDozqbdG2dvsrXhpfo5cfMsZs1NgIjg8RAAN+ZFJfQJuXtk8IAk/ywcs1TMg/IIGNc3sPD65rWBs0rZf54UmaHHoc8RpcnRRYaEJJr/2jplR44fJrMWlm5R5/s1xb2fzSNOKstvlFnQIJCPMSUCxLhP38mJ/CFfy/aRKs/4ocoNkpu
vc8qYyZ/eP7IQUmPN93PmnF23DsbDYeDq8HFVAkaz2Gm8LEA51CugYSQSeOG+QNStv1XeHTNb7QPeL0DZAWIscgyiYXUPOwOgHnXu1CycyZQHrnTa9+0ja2W/KhwjpADsAkMM3PI/UuJdMnRw/PHXYfCQROoK+JbT5Lri4aAGDjqkONX7Q0E
4c80Fvrn9pPlqsiheBTyaDpx7zNKYZ+xFDO/8VlPCDjHZkcR8x8LJo9xhFJyIa2R0Ga21U+lsPZxu01hzKVxLxH1Vbj GgACkdYnFKRe9kmX7cGsoWMRU1GdD7hkcARKBomyZgEeUHRPZGdAtC5grMrbLbwrQpx+moM5sJbsemMMo9231lMt79
goMBygQE1+LA4MBsOzKTqO6RjQyZjpQJiRNxcK8vZzrWWpOfCEloT+7Vt+pGkb3Zyl+e2m6WOgD2SmyuzQGVZApSzgKw7GX5sRPiO1hmF8xuZJytD2PZqt48CijRDvEYnrIDXAzzeE3lOeW5bitduv9UKLusraBjN0AFtAFKUdix8VYK7hluAIQIeHpBeG5AKc4hdGcjQIiFQr1BExH2DAA2lB
eHJRo4+x5l3/YqEkNc8TYEmfbxAzisC5iu2nWhR1XJ5uAeK1mzXnn9iMaafMvD1MvRT6dovCVcr1MNuxwd6AFphYZRsr9V5BZ5cqnf8m1IAPJGbUnBDFuFV3+MJciIGdci1kobv+57aFN75twi09JJeo1pFbFQy4PzrbYadcqVEt6s1glqC
XTPsxKZZ3hYFkZYxSMKIUSADMpStpSwHx3ZckSeTB4x1FFDOqTS/1k1BMtWPEKbmb+PtF2X+UAE/ehnB+WwvZlcfc44VMsx6/AHLP/SQxiSf/VjQKPOqWLZtfvE58Srn/AWSsDax9RCYeqU4SLKnicQCXTV7EDbhgayKcuiqtQGSVYo5BlphI
i3YiiIWA7YpqCqbICLdb8uXVGO3ooveeeOoSRJpK+7Mlnq1hEQTEx10nfde7uhhdvO2Si6SZTRBowq8cqM8q8n8k05TGGZUban2oWle9pKD5g9JICw5kCTQMPUgMaiKw9ucPGJDAG5xGSXCH/CVSLHiZ+hFmJeWoS+6ZM3TwsvtTd1iOUlEb
mkVVr4C4X7EMaiV1LraCbCryh52OxGYcDGZdm9Go8y0cbYl7KH3Q11+THMIr+D7PjImG9LqNgMFZv3lDWvoGquWaSPMWYDbkpKLh6sE3FO4iOxOdmhrOBDweFGw+t+uqjlBespzujzKu3omyKlPEsi37bfFKvonl5bH4+TRzSLnut3MEDWWH
yQwpj5itKudWNGY+DQakL2ydtHwAg2WQgwFSrfCrTN1qyamAmqXDHui94Ozzw5RtY2//PyPapmrNn4fu1tnkHHdNfoRjMqmiwAZk0FlP8vR+IoW5WTlBNpod+LFCzcK9xcsuFP3kFBl4iWLKM3s2lltLTa7q4SJJ8a/quebf1oxnR6dvV/+l
xE1lWQC15hJ1oIZ2KYJKaw2cIlhKNWiJrtaZuQPcLQnffv57TT9UL31WzAjcXPxWMV8qajkcR2S9XNjq9aGBOIQ2NpmqTlPspW9V+hr3iCAOKrVHLjarDIQcQ8oqn7GmAC+s+nZASwm/2ueaXY1lvsGMGjG/2/HMbki+7RGaCKqB9QNph9
S0BtcloTzFC3gTcuTir1NNOqf2d/U8X4JkfRmTEwt+96YGMJ22oUgwBHcof5ddQzB8aGRFS4+kyr5mX/UOSn5W7rxCf3KjQtK7LJeXK75ZeFS8wQQtWpbrZs1jxb4bMd2fw1Rmyczg8Jj71Wh7TaTyIzEHf2SMVlHK3JsaqjVa0ksILTPJEg
gwrgseoNDISJ9hagvSCH+qORLEWyhsxCE1rABJQC1I2qcplYIh8YeFWgbdAyf8b8W9ES5GcLGnvHQH1PwXBpWao7BHm1pLNnBdoWpAArKqnaifhoS+qIrBe06hj5MMcmDWcdyzQGO33Mbrxjx/eXwvueqej96Pp3/C113B0d74a23RHH09
aL0qkrWKKJrrzN5+e6Iq/GIaIcgS7AwJtfZSG0HR+KzXUc/GsCqrvRyzJStzAns3FBlJGmKvQMsHX6UNe+8GZWeaiM6vJs4R52GThuQl8QG+0cFClBZhu6uSKBSSCJW0tlmXKk23EFVeNlxK/G/eHdiIXNP/oFa52/UzDWKiBnZDzTlfXefr
Nq2FNIStDb6IcSipINJQ0lCS9fHL8EQBh2APF8qxj+szPHy3gqsdsxcSMnciUVRQza2lXpmQ7N7En8bu4nJlh5YcLqwTRyU0BwCl6Q5vJmXYKgr1avpvDrYHMegLRT+l8TSIyZVlu3KJgvkgAyQyMGd8sCELuFyxWXVzRdJKey4uTFJRFtAvn
bEEhkU7biY8hKm4DC1VwsdOry/dkOphMsUNjIJ0qSjiHfR9Xsqbf+VHEf92rCPl+z4eCMUyWp2uopb2XJ/9/CbHPSwgdEYfYg1Z6flDJUerKqwz7uKtucFFjleqBEuTRWseSqLU+nFOb4gkr9NM03M7D1KmLfi4i9rIHMJYTNoXLaulWocLsh
S2nJOthmo0kWLfvF6stxKZgLiQayTqAM8+eUR1ARyWhBaARKHK6VR29vFcmJcLJsucrFZVO18yBxkPBaG1AtJTuO7ZaYI7MGqe2S2+95hdQsvSb52eZtAo5MtX1kN9SutnCUIMARMyENFmqm7WxJNKiFA9YYigJgQvzW/+Vwn6R3kPKb81TI
wn//CVBLBwhIbQ4AAJQxABQSwMEFAAIAAgAgZCdWwAAAAAAAAATQEAAC4AIABfX01BQ09TWC9GdXVsLVBvQ3MvLl9jMDItcmVwbGF5LWF0dGFjay50ZXN0LnRzVVQNAAcmYFJpTGBSaWZgUml1eAsAAQT1AQAABFAAAABjYBVj2ZB1
YPBNTFbwD1aIUIACkBgDJxAbMTAwSgNpIJ/R14Eo4BgSEgRhgXWcAOJWNCVMUHEPBgbV5Pxcvc5CgpxUvdzUksSUxJjEqZxfF8+5INzwiNSiVLei/Nx1kFkCQMKWgEUOb6wNLEoMa8kMy+VIakgJ7O4xMBzIWO8gGRGSUlBsZW+fnlqkl5J
ak5qelFirl5+Ubo+BxfUXYxI7kAGaoX6BgYWhtZmlqZGZgZGtZhmWWJOSmZlqZGLobORkamuoYmpqa6JpYm5roWFk4uuqZmro6Wps7mFo6ODgwAUEsHCEBq56/iAAAATQEAAFBLAQIUAxQAAAAAAJVgnVsAAAAAAAAAAAAAAAKACAAAAAA
AAAAAADtQQAAAABGdXVsLVBvQ3MvVVVQNAAdKYFJpSmBSaUpgUml1eAsAAQT1AQAABBQAAABQSwECFAMUAAgACACCYJ1bACRfh+8MAAAyMgAAJQAgAAAAAAAAAAApIFIAAAARnV1bC1Qb0NzL2MwMS1tdWx0aXNpZv1eXBhc3MudGVzdC50
c1VUDQAHJGBSaUxgUml1eAsAAQT1AQAABFAAAABjYBVj2ZB1
```